CATHOLIC UNIVERSITY OF EASTERN AFRICA

PROJECT PROGRESS FOR FINAL YEAR STUDY IN BACHELOR IN COMPUTER SCIENCE

BY

JULIUS MAINGI NJUGUNA ,1062811

PROJECT TITLE

INTERNATIONAL BUS BOOKING SYSTEM

DATE: [JANUARY, 2026]

Methods I Used to Understand the Domain:

To fully understand the problem area, I applied the following methods:

Method	Description
Observation	I booked an SGR train and intercity bus online (Tahmeed Express) to experience the real system workflow.
Interviews	I spoke to bus staff and passengers (my relatives who have used online booking) about ticketing and check-in experiences.
Document Review	I studied booking receipts, SMS confirmations, and station ticket printouts.
Online Research	I reviewed bus company websites and apps (i.e, Madaraka Express, Tahmeed Express, Modern Coast, Easy-Coach, ENA Coach, Dreamline).
Comparative Study	I compared Kenya's systems with other regional or global online bus booking services (e.g. FlixBus, Greyhound).

<u>Document reviews/Observations</u>

Ticket No 16743835 Sold at Nairobi Terminus

Nairobi Terminus → E4 → Mombasa Terminus

DepartureTime: 22:00
the 14th of October, 2025

KSH 1500.00

Seat1 Coach13
ECONOMY

Only valid for carriage on the date
NOT TRANSFERABLE

Name: JULIUS MAINGI NJUGUNA
ID/P NO: 41542848

SerialNo00100001202510142067438 35

TAHMEED EXPRESS

+2540724581902 | +255784818814

MOMBASA — **NAIROBI**

MOMBASA

MoP	PAYMENT GATEWAY	ASSIGNED BUS --
Sub Total	KES 1,000	FRIDAY 17 OCT 2025
Total Paid	KES 1,000	

JULIUS MAINGI NJUGUNA

Tel: 254746627517

ID# 41542848

Reporting:09:15 PM
Departure: 09:45 PM

SERIES	CLASS	SEAT
A	BCLASS	1

Ticket # TAH115818215

Booking Ref: TE1AH5M9ZE

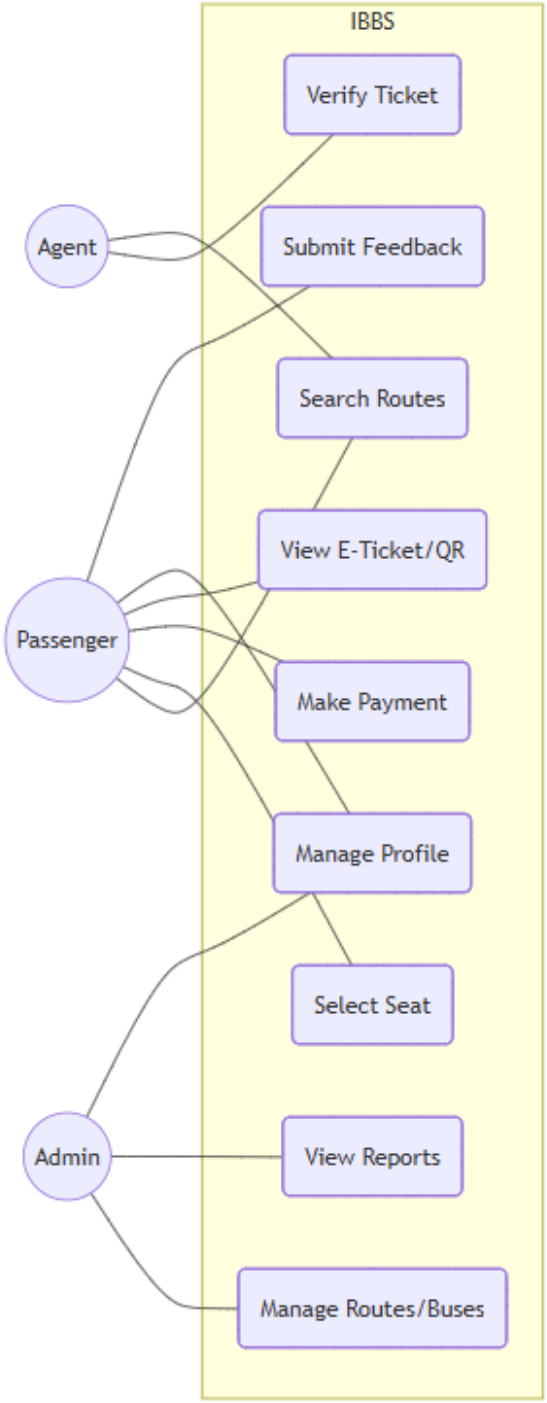
TERMS & CONDITIONS

1. Compensation will not be paid for changes or cancellation of ticket 2. Children above FOUR years must pay full fare

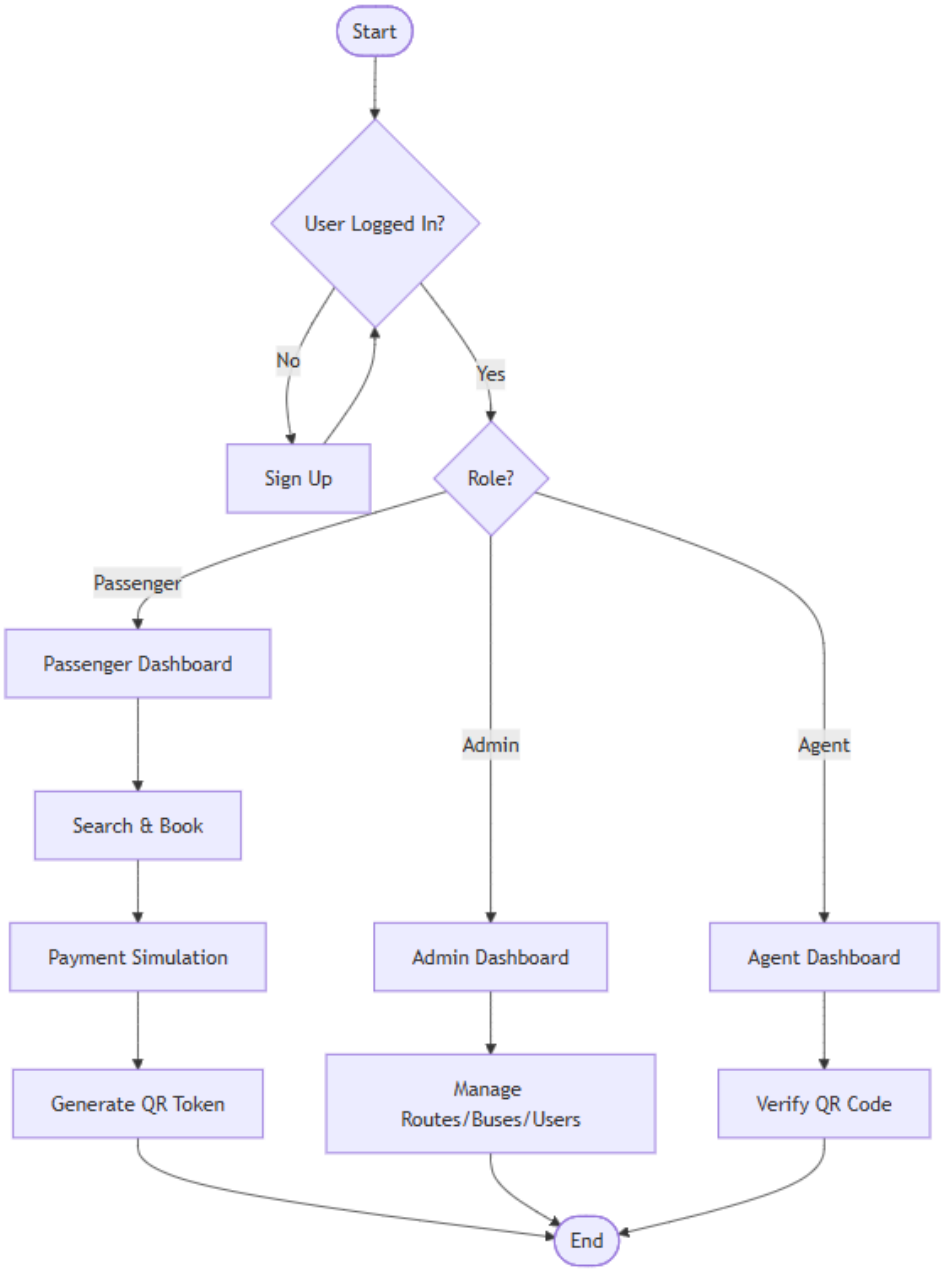
Booked by: **Online** (N/A) | 17th Oct 2025 |
Printed on: 17th Oct 2025 07:08 PM

POWERED BY VOLTIC

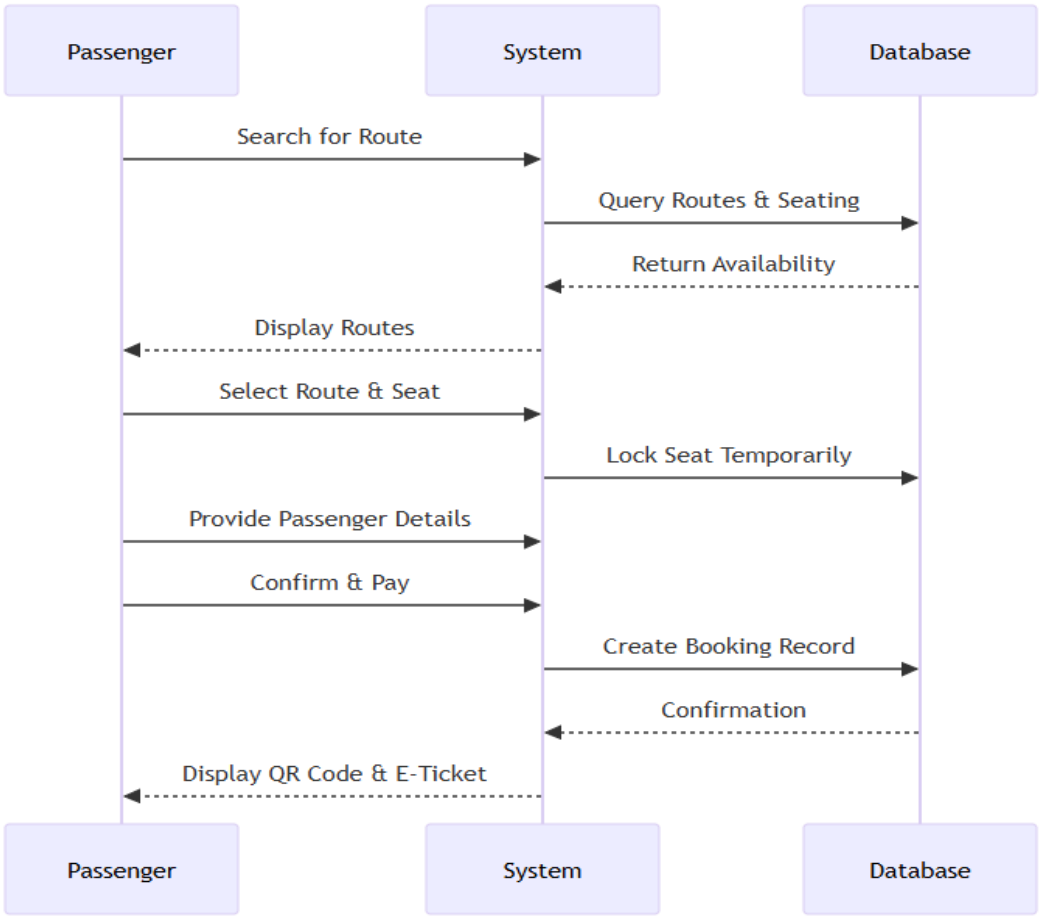
Use Case Diagram



System Flowchart



Process Flow (Booking)



BACKGROUND STUDY

1. Introduction

In recent years, Kenya and neighboring East African countries have embraced digital transformation in transport management. Many transport companies have adopted **online booking systems** that allow passengers to **reserve seats**, **select travel dates**, and **pay using M-Pesa or cards** without visiting physical offices.

However, while these systems have improved convenience for **domestic travel**, challenges remain when it comes to **international routes** (e.g., Kenya–Uganda, Kenya–Tanzania). Systems are often **not interconnected**, passengers must **print tickets physically**, and **border clearance coordination** is largely manual.

1. Passengers Must Print Tickets Physically

Even though passengers **book and pay online**, many transport companies (especially across East Africa) still **require printed tickets** for verification at the bus station.

What this means

- After booking through the company's website or app (e.g., Modern Coast, Mash Poa, or Easy Coach), the passenger receives:
    - A **confirmation message** (SMS or email) with a **booking reference,** booking ID or **ticket number**.
- However, when arriving at the bus station, **they must visit a physical counter** to:
    - Confirm their booking manually from the company's system.
    - Print the actual ticket slip before boarding.

Why this happens

1. **Verification issues** – Some systems at bus stations are not fully integrated with online databases in real time, so staff rely on printed tickets as proof of payment.
2. **Limited infrastructure** – Conductors or drivers might not have handheld devices or scanners to verify digital tickets directly.
3. **Connectivity problems** – Network outages or slow systems can make digital verification unreliable.
4. **Regulatory requirements** – In some cases, transport authorities still require printed tickets for record-keeping, insurance, or audit purposes.

Implications

- Increased **waiting time** for passengers (queues to print).
- Extra **operational costs** (paper, printers, staffing).
- Reduced **environmental sustainability** (paper waste).
- Poor **customer experience**, especially for cross-border travelers who expect fully digital boarding.

  The gap my project will address based on the problems above is:

- To Implement a system that **enables digital ticket verification** (QR codes or e-tickets).

Overview of the Domain Area

The **bus transport sector in Kenya** plays a crucial role in both domestic and regional mobility. With the integration of **digital payments (M-Pesa)** and **online booking**, passengers can now book tickets from anywhere.

However, in **international travel**, such as routes connecting **Nairobi–Kampala** or **Nairobi–Arusha**, operations face unique challenges:

- **Manual check-in** at stations or border points.
- **Physical ticket printing** required for verification.
- Lack of **system integration** across borders.

Thus, the domain area includes:

- **Online ticketing**
- **Seat management**
- **Bus scheduling**
- **Passenger verification**

1. Online Ticketing

This is the core function of the system.
It allows passengers to:

- Browse available routes, dates, and bus operators.
- Select a preferred bus, seat, and travel date.
- Generate an **electronic ticket (e-ticket)** or **QR code** once payment is completed.
- Access their ticket from any device from the website.

Why is this important:
It removes the need to visit a bus station physically and minimizes errors caused by manual booking.

---

2. **Seat Management**

This feature ensures accurate and real-time seat reservation.
It allows:

- Passengers to **view seat availability** in real time.
- Automatic seat locking once a passenger begins a booking.
- Prevention of **double-booking** or **overbooking**.

Why important:
It improves user experience and ensures that bus operators can efficiently plan their trips and capacities.

---

3. **Bus Scheduling**

This manages **bus departures, arrivals, and route planning**.
The system records:

- Departure and arrival times.
- Stops along the route.
- Assigned buses and drivers.

Why important:
It allows managers to monitor fleet usage, optimize time, and ensure punctuality across routes.

---

4. **Passenger Verification**

This component ensures that passengers boarding the bus are **genuine ticket holders**.
It may involve:

- Scanning **QR codes or confirmation codes** at the station.
- Validating tickets using mobile or handheld devices.
- Matching ticket details to passenger identification.

Why important:
It enhances security, prevents ticket fraud, and speeds up the check-in process.

Problem Statement

The "Manual Verification Bottleneck"

Despite the growth of online booking and digital payments, the international bus industry remains slowed down by a **"Last-Mile Manual Bottleneck."** Passengers are still forced to print physical tickets or present confirmation codes that terminal staff must check by hand. This reliance on manual verification causes long queues at boarding, increases the risk of fraud through counterfeit records, and makes it difficult for operators to maintain accurate passenger manifests for cross-border compliance.

Note: While full hardware integration for QR-scanning requires specialized mobile APIs, this project addresses the core challenge by providing a secure digital token framework for automated validation.

Objectives

Main Objective

To build a digital bus booking and verification system that replaces manual ticketing with a secure digital token framework.

Specific Objectives

1. **Online Travel Portal**: To develop a web-based platform where passengers can search for international routes, compare costs, and select seats in real-time.
2. **Digital Token Generation**: To design a system that generates unique digital identifiers (tokens) for every booking, ensuring each ticket is one-of-a-kind and tamper-proof.
3. **Simulated Verification Framework**: To implement the database logic required for automated ticket validation, allowing staff to verify boarding passes instantly without manual logs.

4. **Comprehensive Manifest Management**: To create a database that captures detailed passenger information (Name, Age, ID) to satisfy cross-border travel requirements.
5. **Admin Control Center**: To provide bus operators with a centralized dashboard for managing fleets, travel schedules, and real-time occupancy.

Room for Improvement (Future Phase / Out of Scope)

6. **Physical QR Scanner API**: Integrating specialized mobile camera APIs or hardware scanners to physically "read" the generated tokens at the bus door.
7. **Live Notification Gateways**: Integrating SMS and Email APIs to send automated travel alerts and reminders (currently simulated within the UI).
8. **Automated Payment Settlement**: Integrating the M-Pesa Daraja API or Card Gateways for real-time financial reconciliation.

For an improvement I will make a page called verify_ticket.php where a staff member just pastes the token, and it says **"VALID: Seat 5A, John Doe"** or **"INVALID"** to validate the tokens.

IDENTIFIED USERS AND THEIR ROLES

1. Administrator (ADMIN)

The highest authority in the system. The Admin manages the business logic and fleet.

- **User Management**: Can create, edit, or delete accounts for Agents and Passengers.
- **Fleet Management**: Registers new buses and assigns/de-assigns drivers to them.
- **Schedule Management**: Responsible for creating and updating international travel routes and setting ticket prices.
- **Business Intelligence**: Accesses high-level reports, including **Revenue Reports**, **Agent Performance**, and **Bus Occupancy** trends.
- **System Oversight**: Can view and moderate all customer feedback.

2. Booking Agent (AGENT)

Staff members stationed at terminals or travel offices who facilitate the booking process.

- **Walk-in Bookings**: Authorized to book tickets for customers who visit the station physically.
- **Passenger Support**: Assists travellers with schedule information, seat availability, and booking cancellations.
- **Seat Monitoring**: Monitors real-time bus occupancy to prevent overbooking.
- **Identity Registration**: Can register new passengers into the system to facilitate their first booking.

3. Passenger (PASSENGER)

The primary customer or traveller using the portal.

- **Self-Service Booking**: Can search for routes, compare fares, and select their preferred seats.
- **Personal Dashboard**: Accesses their personal travel history and profile.
- **Digital Ticketing**: Retrieves and views their **Digital Travel Passes** (QR-code tokens) for boarding.
- **Feedback & Ratings**: Can rate their travel experience and leave comments about specific buses or routes.

System Actors (Managed but non-login)

While these individuals do not log into the system themselves, they are key "actors" tracked by the software:

- **Driver**: A registered professional assigned to specific buses. The system tracks their National ID, licensing status, and contact details to ensure every trip has a designated operator.

DATABASE CREATION SCRIPTS

```
DROP DATABASE IF EXISTS IBBS_PROTOTYPE;
CREATE DATABASE IBBS_PROTOTYPE;
USE IBBS_PROTOTYPE;
```

1. USERS TABLE

```
CREATE TABLE users (
user_id INT(11) NOT NULL AUTO_INCREMENT,
first_name VARCHAR(50) NOT NULL,
last_name VARCHAR(50) NOT NULL,
email VARCHAR(100) NOT NULL UNIQUE,
phone_number VARCHAR(15) NOT NULL UNIQUE,
password VARCHAR(255) NOT NULL,
role ENUM('PASSENGER','ADMIN','AGENT') DEFAULT 'PASSENGER',
created_at TIMESTAMP NOT NULL DEFAULT current_timestamp(),
PRIMARY KEY (user_id)
);
```

2. DRIVERS TABLE

```
CREATE TABLE drivers (
driver_id INT(11) NOT NULL AUTO_INCREMENT,
national_id VARCHAR(20) NOT NULL UNIQUE,
full_name VARCHAR(100) NOT NULL,
phone VARCHAR(15) NOT NULL UNIQUE,
email VARCHAR(100) NOT NULL UNIQUE,
PRIMARY KEY (driver_id)
```

```
);
```

3. BUSES TABLE

```
CREATE TABLE buses (
bus_id INT(11) NOT NULL AUTO_INCREMENT,
reg_no VARCHAR(20) NOT NULL UNIQUE,
bus_name VARCHAR(50) DEFAULT NULL,
max_passengers INT(11) NOT NULL,
seat_layout VARCHAR(20) NOT NULL,
driver_id INT(11) DEFAULT NULL,
PRIMARY KEY (bus_id),
FOREIGN KEY (driver_id) REFERENCES drivers (driver_id) ON DELETE SET NULL
);
```

4. ROUTES TABLE

```
CREATE TABLE routes (
route_id INT(11) NOT NULL AUTO_INCREMENT,
from_location VARCHAR(100) NOT NULL,
to_location VARCHAR(100) NOT NULL,
departure_date DATE NOT NULL,
departure_time TIME NOT NULL,
cost DECIMAL(10,2) NOT NULL,
bus_id INT(11) NOT NULL,
status ENUM('SCHEDULED','COMPLETED','CANCELLED') DEFAULT 'SCHEDULED',
PRIMARY KEY (route_id),
FOREIGN KEY (bus_id) REFERENCES buses (bus_id)
);
```

5. BOOKINGS TABLE

```
CREATE TABLE bookings (
booking_id INT AUTO_INCREMENT PRIMARY KEY,
booking_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
seat_number VARCHAR(10) NOT NULL,
booking_status ENUM('CONFIRMED','CANCELLED','PAID','CHECKED_IN') DEFAULT 'PAID',
qr_token VARCHAR(255) UNIQUE,
user_id INT NOT NULL,
route_id INT NOT NULL,
bus_id INT NOT NULL,
passenger_name VARCHAR(255) NOT NULL DEFAULT '',
passenger_age INT NOT NULL DEFAULT 0,
passenger_id_number VARCHAR(50) NOT NULL DEFAULT '',
passenger_dob DATE DEFAULT NULL,
FOREIGN KEY (user_id) REFERENCES users(user_id),
FOREIGN KEY (route_id) REFERENCES routes(route_id) ON DELETE CASCADE,
FOREIGN KEY (bus_id) REFERENCES buses(bus_id),
UNIQUE KEY unique_booking (route_id, seat_number)
);
```

6. FEEDBACK TABLE

```
CREATE TABLE feedback (
feedback_id INT(11) NOT NULL AUTO_INCREMENT,
rating INT(11) DEFAULT 5,
comments TEXT NOT NULL,
feedback_date DATE NOT NULL,
user_id INT(11) NOT NULL,
```

```
bus_id INT(11) NOT NULL,
route_id INT(11) NOT NULL,
submitted_at TIMESTAMP NOT NULL DEFAULT current_timestamp(),
PRIMARY KEY (feedback_id),
FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE,
FOREIGN KEY (bus_id) REFERENCES buses (bus_id),
FOREIGN KEY (route_id) REFERENCES routes (route_id) ON DELETE CASCADE
);
```

DATA INSERTION SCRIPTS

Inserting ADMIN Users

```
INSERT INTO users (first_name, last_name, email, phone_number, password, role) VALUES
('Alice', 'Kamau', 'alice1@gmail.com', '0700000000',
'$2y$10$6AP9mQLbsAlcc2yDxdX3.Oed376Faioz4iG2NG7Z5ZTsEOQU8huQe', 'ADMIN'),
('Bob', 'Ochieng', 'bob2@gmail.com', '0700000001',
'$2y$10$eOws.VlRQIGeg7axR6s9pOcAEHNSQ7ogTbPqG4uGzsnXQJ1jf2GUK', 'ADMIN'),
('Charlie', 'Kipkorir', 'charlie3@gmail.com', '0700000002',
'$2y$10$nnpgJAAK5jr6CwCNQbFJTuRXCZZY017LneYwnjbpX2IhOa1aHKZqW', 'ADMIN'),
('David', 'Maina', 'david4@gmail.com', '0700000003',
'$2y$10$KPSYO2TIhGX2uVCcycR08uyYkqVoQMnq7t1xWLGIP0j8d5Hhz7y32', 'ADMIN'),
('Eve', 'Wanjiku', 'eve5@gmail.com', '0700000004',
'$2y$10$07KWtn7qvTnlWR4AWJprzujaiG4W0QuNot90mOJaD61jwkIHcBOVS', 'ADMIN'),
('Frank', 'Otieno', 'frank6@gmail.com', '0700000005',
'$2y$10$Dw4bGCBQoh6he42VVOKyqegyTzNd81KDtP8J0hg3r79pAOfZWll/e', 'ADMIN'),
('Grace', 'Achieng', 'grace7@gmail.com', '0700000006',
'$2y$10$Kf8pw34A0CbitzLb2W1uhOrJuy3Jpz8YaBkHZnFrCGUEWvToaukuC', 'ADMIN'),
('Hank', 'Musyoka', 'hank8@gmail.com', '0700000007',
'$2y$10$7AfpBfCzt0V0ID.iVXutLOdQjeT6YfQQTz7toR776amoEqHSKpApK', 'ADMIN'),
```

('Ivy', 'Njeri', 'ivy9@gmail.com', '0700000008',
'$2y$10$l7XO37p6M77kIXQmFT/7au5acAaWJyqXPueFCs6QEDK2rT7iAE10u', 'ADMIN'),

('Jack', 'Mutua', 'jack10@gmail.com', '0700000009',
'$2y$10$faynGAGhm8/w.YVgf42B1eSL7Kta9AdKvGCSz93ZfhT/Oge6Wu2Z2', 'ADMIN');

Inserting AGENT Users

INSERT INTO users (first_name, last_name, email, phone_number, password, role) VALUES

('Karen', 'Njoroge', 'karen1@gmail.com', '0711000000',
'$2y$10$vorDX2PZQq6CNyKhnEd6muNccbOp0/kAsmf/VoOvsdL9VixyFckha', 'AGENT'),

('Leo', 'Mwangi', 'leo2@gmail.com', '0711000001',
'$2y$10$n63JIZe9OeTTLHSgl9HrEuSQsam528/xT2XNpUo/jhNka2HVBWrYK', 'AGENT'),

('Mia', 'Odhiambo', 'mia3@gmail.com', '0711000002',
'$2y$10$e2UZmFWu0tvbMmaWdt/vtObHNqAnVEaY3JuZL2q6rKn9gDYQWXcPW', 'AGENT'),

('Noah', 'Kimani', 'noah4@gmail.com', '0711000003',
'$2y$10$OtaT8IBKXLzF7TwwxVjmpe8UyX7iXI2a991g2Bnf7BUbt4tjApzMu', 'AGENT'),

('Olivia', 'Chebet', 'olivia5@gmail.com', '0711000004',
'$2y$10$Vajn3ZpLHhurmfDz/CsvluQznpN7kTYjjk.DxI9ktS4VJt9SicYmm', 'AGENT'),

('Paul', 'Kariuki', 'paul6@gmail.com', '0711000005',
'$2y$10$QTQGUhYKwJv9krYSEI/sku0vRIsACDQ2eE3FD3Ac07/b8tYf2C9m6', 'AGENT'),

('Quinn', 'Awere', 'quinn7@gmail.com', '0711000006',
'$2y$10$LntrFhaxjyTKyGEShGKFsuY/j7E5McloZPjzKoDRDozZjMmnaRjqS', 'AGENT'),

('Ryan', 'Omondi', 'ryan8@gmail.com', '0711000007',
'$2y$10$/Y6W3rm6Ug1L5sWVHCeQn.QwOFX0brsk7jyUpxNXLus.aayOwA1SO', 'AGENT'),

('Sarah', 'Wambui', 'sarah9@gmail.com', '0711000008',
'$2y$10$ZTnq0FaW5YapQmPJCahjredJ3fs.VlquxaIxWfSQhS7u82DBdGBOK', 'AGENT'),

('Tom', 'Ndegwa', 'tom10@gmail.com', '0711000009',
'$2y$10$0L.4AyomYevMyybNyBoGue4ZEUqn8I1S..h8MV7jyyx1Mi5sW3S5G', 'AGENT');

Inserting PASSENGER Users

INSERT INTO users (first_name, last_name, email, phone_number, password, role) VALUES

('Uma', 'Abdi', 'uma1@gmail.com', '0722000000',
'$2y$10$1YvBeMq4Ot4OIzluMOkL8eSPlDm/vf1RvJDhD/S94gnKL7hVgKHx2', 'PASSENGER'),

('Vin', 'Ndlovu', 'vin2@gmail.com', '0722000001',
'$2y$10$NHV5qfiqIxq8CX08oMIAzu1HUU1aAzAg/VeCszLban9to86DGxr7O', 'PASSENGER'),

('Will', 'Chamele', 'will3@gmail.com', '0722000002',
'$2y$10$isNOgS0qXW3/jaKOHK/5aeMDajNAnJPxU5NhcAzUdRPr5NzdSReiu', 'PASSENGER'),

('Xena', 'Tesfaye', 'xena4@gmail.com', '0722000003',
'$2y$10$KY1KRlHaK7fU.fPjWQzrGehf4EtXzqD6cZKOU55Ysbwwv9ojHR3Fi', 'PASSENGER'),

('Yara', 'Mensah', 'yara5@gmail.com', '0722000004',
'$2y$10$/cYOmtVIWUga.v0oLa6kEukgJT922bVzTMjCmkLuuBLK6WlPUC6Fi', 'PASSENGER'),

('Zac', 'Diallo', 'zac6@gmail.com', '0722000005',
'$2y$10$uI27jAHOQfMptYojSuWiQ.U/ZrbkbPRZsk8wG0IQr8CXWljPEWGv2', 'PASSENGER'),

('Adam', 'Keita', 'adam7@gmail.com', '0722000006',
'$2y$10$cKzfWlu7dfxQaVZ9h4YdOumimgSVLp8isYQzHvspOMom9.RUuUAhG', 'PASSENGER'),

('Bella', 'Sow', 'bella8@gmail.com', '0722000007',
'$2y$10$CE03fm6meXEWMKeL48fXsunPESkwcP.krgkbZSUDv.v2.YMioHu6q', 'PASSENGER'),

('Chris', 'Traore', 'chris9@gmail.com', '0722000008',
'$2y$10$J/xI66qvwJoFOei8mACNSueIhH9xVv6va7E.4tgaolFxQLyx86xvW', 'PASSENGER'),

('Drake', 'Kone', 'drake10@gmail.com', '0722000009',
'$2y$10$FjK8MDssTU5GMnetQ9FAIe1NIK/yi0JNY7bIbqAoxi2i9MouNqxfS', 'PASSENGER');

INSERT DRIVERS

INSERT INTO drivers (national_id, full_name, phone, email) VALUES

('ID00000001', 'John Mwangi', '0733000000', 'john.mwangi@gmail.com'),

('ID00000002', 'Samuel Okello', '0733000001', 'samuel.okello@gmail.com'),

('ID00000003', 'David Mengistu', '0733000002', 'david.mengistu@gmail.com'),

('ID00000004', 'Mohammed Hassan', '0733000003', 'mohammed.hassan@gmail.com'),

('ID00000005', 'Peter Kamau', '0733000004', 'peter.kamau@gmail.com'),

('ID00000006', 'James Omondi', '0733000005', 'james.omondi@gmail.com'),

('ID00000007', 'Benson Kiprop', '0733000006', 'benson.kiprop@gmail.com'),

('ID00000008', 'Charles Odhiambo', '0733000007', 'charles.odhiambo@gmail.com'),

('ID00000009', 'Joseph Kariuki', '0733000008', 'joseph.kariuki@gmail.com'),

('ID00000010', 'Thomas Njoroge', '0733000009', 'thomas.njoroge@gmail.com'),

('ID00000011', 'Emmanuel Abebe', '0733000010', 'emmanuel.abebe@gmail.com'),

('ID00000012', 'Isaac Tesfaye', '0733000011', 'isaac.tesfaye@gmail.com'),

('ID00000013', 'Gabriel Selassie', '0733000012', 'gabriel.selassie@gmail.com'),

('ID00000014', 'Michael Afewerki', '0733000013', 'michael.afewerki@gmail.com'),

('ID00000015', 'Daniel Tekle', '0733000014', 'daniel.tekle@gmail.com'),

('ID00000016', 'Richard Mwanza', '0733000015', 'richard.mwanza@gmail.com'),

('ID00000017', 'Patrick Mutua', '0733000016', 'patrick.mutua@gmail.com'),

('ID00000018', 'Stephen Musyoka', '0733000017', 'stephen.musyoka@gmail.com'),

('ID00000019', 'George Otieno', '0733000018', 'george.otieno@gmail.com'),

('ID00000020', 'Edward Wanyama', '0733000019', 'edward.wanyama@gmail.com'),

('ID00000021', 'Brian Kibet', '0733000020', 'brian.kibet@gmail.com'),

('ID00000022', 'Kevin Cheruiyot', '0733000021', 'kevin.cheruiyot@gmail.com'),

('ID00000023', 'Dennis Kemboi', '0733000022', 'dennis.kemboi@gmail.com'),

('ID00000024', 'Alex Rotich', '0733000023', 'alex.rotich@gmail.com'),

('ID00000025', 'Felix Langat', '0733000024', 'felix.langat@gmail.com'),

('ID00000026', 'Victor Ochieng', '0733000025', 'victor.ochieng@gmail.com'),

('ID00000027', 'Collins Okeyo', '0733000026', 'collins.okeyo@gmail.com'),

('ID00000028', 'Fredrick Owino', '0733000027', 'fredrick.owino@gmail.com'),

('ID00000029', 'Walter Aketch', '0733000028', 'walter.aketch@gmail.com'),

('ID00000030', 'Moses Odongo', '0733000029', 'moses.odongo@gmail.com');

INSERT BUSES

INSERT INTO buses (reg_no, bus_name, max_passengers, seat_layout, driver_id) VALUES

('KWI 435Z', 'Wema Executive 1', 40, '2x2', 1),

('KIP 893K', 'Wema Executive 2', 40, '2x2', 2),

('KHN 719W', 'Wema Executive 3', 40, '2x2', 3),

('KOY 346P', 'Wema Executive 4', 40, '2x2', 4),

('UPP 520A', 'Wema Executive 5', 40, '2x2', 5),

('UUH 503W', 'Wema Executive 6', 40, '2x2', 6),

('T 435 VWM', 'Wema Executive 7', 40, '2x2', 7),

('T 155 UII', 'Wema Executive 8', 40, '2x2', 8),

('RLR 718 H', 'Wema Executive 9', 40, '2x2', 9),

('SIJ 854 GP', 'Wema Executive 10', 40, '2x2', 10),

('KJI 112Z', 'Wema Executive 11', 40, '2x2', 11),

('KOT 120F', 'Wema Executive 12', 40, '2x2', 12),

('KJX 893M', 'Wema Executive 13', 40, '2x2', 13),

('KAL 785Y', 'Wema Executive 14', 40, '2x2', 14),

('UAP 465F', 'Wema Executive 15', 40, '2x2', 15),

('UTK 299E', 'Wema Executive 16', 40, '2x2', 16),

('T 175 CCH', 'Wema Executive 17', 40, '2x2', 17),

('T 676 VKM', 'Wema Executive 18', 40, '2x2', 18),

('RQI 605 S', 'Wema Executive 19', 40, '2x2', 19),

('ENM 318 GP', 'Wema Executive 20', 40, '2x2', 20),

('KHJ 537G', 'Wema Executive 21', 40, '2x2', 21),

('KZQ 699X', 'Wema Executive 22', 40, '2x2', 22),

('KUI 133D', 'Wema Executive 23', 40, '2x2', 23),

('KIS 653J', 'Wema Executive 24', 40, '2x2', 24),

('UIH 545W', 'Wema Executive 25', 40, '2x2', 25),

('UOF 319L', 'Wema Executive 26', 40, '2x2', 26),

('T 788 UOC', 'Wema Executive 27', 40, '2x2', 27),

('T 909 NSY', 'Wema Executive 28', 40, '2x2', 28),

('RFA 598 Q', 'Wema Executive 29', 40, '2x2', 29),

('YET 839 GP', 'Wema Executive 30', 40, '2x2', 30);

INSERT ROUTES (International)

INSERT INTO routes (from_location, to_location, departure_date, departure_time, cost, bus_id) VALUES

('Nairobi, Kenya', 'Kampala, Uganda', '2026-04-07', '12:00:00', 3500, 1),

('Kampala, Uganda', 'Nairobi, Kenya', '2026-06-14', '15:10:00', 3500, 2),

('Nairobi, Kenya', 'Arusha, Tanzania', '2026-11-25', '14:10:00', 2500, 3),

('Arusha, Tanzania', 'Nairobi, Kenya', '2026-09-20', '20:40:00', 2500, 4),

('Johannesburg, SA', 'Asmara, Eritrea', '2026-05-22', '13:30:00', 25000, 5),

('Mombasa, Kenya', 'Dar es Salaam, Tanzania', '2026-10-05', '17:40:00', 4000, 6),

('Kigali, Rwanda', 'Kampala, Uganda', '2026-11-03', '13:40:00', 2000, 7),

('Bujumbura, Burundi', 'Kigali, Rwanda', '2026-03-18', '14:10:00', 1500, 8),

('Addis Ababa, Ethiopia', 'Nairobi, Kenya', '2026-08-18', '20:00:00', 6000, 9),

('Lusaka, Zambia', 'Dar es Salaam, Tanzania', '2026-06-12', '12:40:00', 8000, 10),

('Nairobi, Kenya', 'Kampala, Uganda', '2026-11-15', '07:10:00', 3500, 11),

('Kampala, Uganda', 'Nairobi, Kenya', '2026-09-01', '10:40:00', 3500, 12),

('Nairobi, Kenya', 'Arusha, Tanzania', '2026-10-05', '17:10:00', 2500, 13),

('Arusha, Tanzania', 'Nairobi, Kenya', '2026-01-16', '20:20:00', 2500, 14),

('Johannesburg, SA', 'Asmara, Eritrea', '2026-05-25', '10:40:00', 25000, 15),

('Mombasa, Kenya', 'Dar es Salaam, Tanzania', '2026-01-20', '20:10:00', 4000, 16),

('Kigali, Rwanda', 'Kampala, Uganda', '2026-04-07', '06:50:00', 2000, 17),

('Bujumbura, Burundi', 'Kigali, Rwanda', '2026-03-26', '06:50:00', 1500, 18),

('Addis Ababa, Ethiopia', 'Nairobi, Kenya', '2026-10-26', '10:20:00', 6000, 19),

('Lusaka, Zambia', 'Dar es Salaam, Tanzania', '2026-04-12', '15:10:00', 8000, 20),

('Nairobi, Kenya', 'Kampala, Uganda', '2026-09-26', '17:20:00', 3500, 21),

('Kampala, Uganda', 'Nairobi, Kenya', '2026-10-24', '16:10:00', 3500, 22),

('Nairobi, Kenya', 'Arusha, Tanzania', '2026-09-05', '09:30:00', 2500, 23),

('Arusha, Tanzania', 'Nairobi, Kenya', '2026-11-04', '09:10:00', 2500, 24),

('Johannesburg, SA', 'Asmara, Eritrea', '2026-08-01', '18:10:00', 25000, 25),

('Mombasa, Kenya', 'Dar es Salaam, Tanzania', '2026-09-09', '08:30:00', 4000, 26),

('Kigali, Rwanda', 'Kampala, Uganda', '2026-09-11', '20:40:00', 2000, 27),

('Bujumbura, Burundi', 'Kigali, Rwanda', '2026-12-07', '20:30:00', 1500, 28),

('Addis Ababa, Ethiopia', 'Nairobi, Kenya', '2026-08-11', '12:20:00', 6000, 29),

('Lusaka, Zambia', 'Dar es Salaam, Tanzania', '2026-06-20', '10:50:00', 8000, 30);

INSERT BOOKINGS

INSERT INTO bookings (booking_time, seat_number, booking_status, qr_token, user_id, route_id, bus_id) VALUES

('2026-06-05 20:00:00', '2A', 'PAID', 'e6233292db93c159fca80cd558ffbfd9', 22, 2, 2),

('2026-10-03 15:00:00', '3A', 'PAID', '0db1515fc46edd6e578051bcd29375b6', 23, 3, 3),

('2026-02-16 20:00:00', '4A', 'PAID', '730e6e438d5f966a527f4f538f132003', 24, 4, 4),

('2026-04-26 14:00:00', '5A', 'PAID', '66fc8bc02fd8ac40242e137f1830bfb9', 25, 5, 5),

('2026-03-27 12:00:00', '6A', 'PAID', '622d6ca0678c67e7fdd6a7d368465fa4', 26, 6, 6),

('2026-02-04 13:00:00', '7A', 'PAID', '26f3029aeed588071f58b38de5757653', 27, 7, 7),

('2026-09-05 20:00:00', '8A', 'PAID', 'dc5486d63381738a637789a92b8085fa', 28, 8, 8),

('2026-04-20 16:00:00', '9A', 'PAID', '71b059ece856b4830ff8068af95eec29', 29, 9, 9),

('2026-06-05 15:00:00', '10A', 'PAID', '4477b9b52f1ae4f00c21a4ea55af147a', 30, 10, 10),

('2026-12-24 10:00:00', '11A', 'PAID', '8a51b1f62ed3c7abd8fd96a67b744c63', 21, 11, 11),

('2026-07-03 19:00:00', '12A', 'PAID', 'eabec05090e648641aca30b6dc9dde54', 22, 12, 12),

('2026-09-18 09:00:00', '13A', 'PAID', '0688f64043a983e55793317de1d00730', 23, 13, 13),

('2026-06-20 14:00:00', '14A', 'PAID', '61d1e57c0708a4edc51066ae81067bac', 24, 14, 14),

('2026-06-19 17:00:00', '15A', 'PAID', '0d5ba3994aa82106ff5e852aa841f395', 25, 15, 15),

('2026-04-14 08:00:00', '16A', 'PAID', '52a4dd281e934256937909057dad588c', 26, 16, 16),

('2026-11-11 12:00:00', '17A', 'PAID', '32fe983757e7eb28580a7a519bc5cd7a', 27, 17, 17),

('2026-01-11 10:00:00', '18A', 'PAID', '373db2a35abc3e3b8b1f6a9eaf88b9bf', 28, 18, 18),

('2026-08-20 19:00:00', '19A', 'PAID', '31400a08f0666ae9067b670816f3a7b7', 29, 19, 19),

('2026-06-01 08:00:00', '20A', 'PAID', '747d0627ee743985911357a1bb6c2185', 30, 20, 20),

('2026-07-14 16:00:00', '21A', 'PAID', 'da61c9e0d991ec6df279d64349ac43b8', 21, 21, 21),

('2026-06-22 14:00:00', '22A', 'PAID', '8d9c0cd02e567801f2a89bcdbd17a720', 22, 22, 22),

('2026-08-24 12:00:00', '23A', 'PAID', '99ac1e189752a4d8b44304de6cd6bbe9', 23, 23, 23),

('2026-07-22 14:00:00', '24A', 'PAID', '98cd95edd59baf6a2333c93be1ecd4b8', 24, 24, 24),

('2026-11-18 14:00:00', '25A', 'PAID', '7c100301a8e2ff2b11e6df1d95730ba5', 25, 25, 25),

('2026-01-05 17:00:00', '26A', 'PAID', '149cf7b391f41ebd4e0151b3c1619c62', 26, 26, 26),

('2026-06-23 16:00:00', '27A', 'PAID', '46f3d5ae88f84387aa08bdf6afd424ed', 27, 27, 27),

('2026-10-01 11:00:00', '28A', 'PAID', 'a23cc6ba5f5ab6e6b4d1d044f30a245d', 28, 28, 28),

('2026-08-04 08:00:00', '29A', 'PAID', '7f89ebecb0dc36b7d2f55fd2480667e1', 29, 29, 29),

('2026-04-21 13:00:00', '30A', 'PAID', '5095b52c5c74e2c358469a1b4fd6755b', 30, 30, 30),

('2026-09-19 14:00:00', '31A', 'PAID', 'e6b951a4a4eb2d2543f08052046565d1', 21, 1, 1);

INSERT FEEDBACK

INSERT INTO feedback (rating, comments, feedback_date, user_id, bus_id, route_id) VALUES

(4, 'Charging ports were not working, please fix.', '2026-12-11', 22, 2, 2),

(3, 'Charging ports were not working, please fix.', '2026-08-04', 23, 3, 3),

(5, 'Great service, very comfortable bus.', '2026-03-25', 24, 4, 4),

(4, 'Love the new buses, very modern and fresh.', '2026-12-25', 25, 5, 5),

(5, 'Bus was a bit late but the ride was okay.', '2026-02-08', 26, 6, 6),

(5, 'Excellent staff and safe driving. Will book again.', '2026-07-17', 27, 7, 7),

(5, 'The driver drove carefully, I felt safe throughout.', '2026-06-02', 28, 8, 8),

(5, 'Best travel experience I have had in a long time.', '2026-06-28', 29, 9, 9),

(5, 'Excellent staff and safe driving. Will book again.', '2026-09-26', 30, 10, 10),

(3, 'Enjoyed the free Wi-Fi, it was surprisingly fast.', '2026-08-09', 21, 11, 11),

(3, 'The snacks provided were a nice touch.', '2026-09-17', 22, 12, 12),

(5, 'Great service, very comfortable bus.', '2026-07-18', 23, 13, 13),

(4, 'Enjoyed the free Wi-Fi, it was surprisingly fast.', '2026-03-17', 24, 14, 14),

(3, 'Too many stops along the way made the trip longer.', '2026-03-13', 25, 15, 15),

(3, 'Booking process was easy and the bus was on time.', '2026-04-11', 26, 16, 16),

(3, 'Too many stops along the way made the trip longer.', '2026-01-15', 27, 17, 17),

(5, 'The bus was noisy, could not sleep well.', '2026-09-15', 28, 18, 18),

(5, 'Booking process was easy and the bus was on time.', '2026-04-16', 29, 19, 19),

(5, 'Bus was a bit late but the ride was okay.', '2026-07-14', 30, 20, 20),

(3, 'Booking process was easy and the bus was on time.', '2026-03-27', 21, 21, 21),

(5, 'Excellent staff and safe driving. Will book again.', '2026-01-11', 22, 22, 22),

(4, 'The journey was smooth and the driver was professional.', '2026-08-25', 23, 23, 23),

(3, 'Enjoyed the free Wi-Fi, it was surprisingly fast.', '2026-06-09', 24, 24, 24),

(4, 'Customer service was helpful when I needed to change my seat.', '2026-02-18', 25, 25, 25),

(5, 'Great service, very comfortable bus.', '2026-04-05', 26, 26, 26),

(4, 'The bus was noisy, could not sleep well.', '2026-12-26', 27, 27, 27),

(4, 'Bus was a bit late but the ride was okay.', '2026-10-01', 28, 28, 28),

(5, 'The journey was smooth and the driver was professional.', '2026-07-06', 29, 29, 29),

(3, 'Arrived at the destination ahead of schedule!', '2026-06-20', 30, 30, 30),

(4, 'Love the new buses, very modern and fresh.', '2026-07-26', 21, 1, 1);

MY SQL CLI SNAPSHOTS

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| ibbs               |
| ibbs_prototype     |
| information_schema |
| mysql              |
| performance_schema |
| phpmyadmin         |
| test               |
| wema_transport     |
+--------------------+
8 rows in set (0.16 sec)

mysql> use ibbs_prototype;
Database changed
mysql> show tables;
+-------------------------+
| Tables_in_ibbs_prototype |
+-------------------------+
| bookings                |
| buses                   |
| drivers                 |
| feedback                |
| routes                  |
| users                   |
+-------------------------+
6 rows in set (0.01 sec)
```

```
mysql> desc bookings;
+--------------------+-----------------------------------------------+------+-----+--------------------+----------------+
| Field              | Type                                          | Null | Key | Default            | Extra          |
+--------------------+-----------------------------------------------+------+-----+--------------------+----------------+
| booking_id         | int(11)                                       | NO   | PRI | NULL               | auto_increment |
| booking_time       | datetime                                      | NO   |     | current_timestamp() |                |
| seat_number        | varchar(10)                                   | NO   |     | NULL               |                |
| booking_status     | enum('CONFIRMED','CANCELLED','PAID','CHECKED_IN') | YES  |     | PAID               |                |
| qr_token           | varchar(255)                                  | YES  |     | NULL               |                |
| user_id            | int(11)                                       | NO   | MUL | NULL               |                |
| route_id           | int(11)                                       | NO   | MUL | NULL               |                |
| bus_id             | int(11)                                       | NO   | MUL | NULL               |                |
| passenger_name     | varchar(255)                                  | NO   |     |                    |                |
| passenger_age      | int(11)                                       | NO   |     | 0                  |                |
| passenger_dob      | date                                          | YES  |     | NULL               |                |
| passenger_id_number | varchar(50)                                   | NO   |     |                    |                |
+--------------------+-----------------------------------------------+------+-----+--------------------+----------------+
12 rows in set (0.01 sec)
```

```
mysql> desc buses;
+----------------+-------------+------+-----+---------+----------------+
| Field          | Type        | Null | Key | Default | Extra          |
+----------------+-------------+------+-----+---------+----------------+
| bus_id         | int(11)     | NO   | PRI | NULL    | auto_increment |
| reg_no         | varchar(20) | NO   | UNI | NULL    |                |
| bus_name       | varchar(50) | YES  |     | NULL    |                |
| max_passengers | int(11)     | NO   |     | NULL    |                |
| seat_layout    | varchar(20) | NO   |     | NULL    |                |
| driver_id      | int(11)     | YES  | MUL | NULL    |                |
+----------------+-------------+------+-----+---------+----------------+
6 rows in set (0.06 sec)

mysql> desc drivers;
+-------------+--------------+------+-----+---------+----------------+
| Field       | Type         | Null | Key | Default | Extra          |
+-------------+--------------+------+-----+---------+----------------+
| driver_id   | int(11)      | NO   | PRI | NULL    | auto_increment |
| national_id | varchar(20)  | NO   | UNI | NULL    |                |
| full_name   | varchar(100) | NO   |     | NULL    |                |
| phone       | varchar(15)  | NO   | UNI | NULL    |                |
| email       | varchar(100) | NO   | UNI | NULL    |                |
+-------------+--------------+------+-----+---------+----------------+
5 rows in set (0.06 sec)
```

```
mysql> desc feedback;
+---------------+-----------+------+-----+--------------------+----------------+
| Field         | Type      | Null | Key | Default            | Extra          |
+---------------+-----------+------+-----+--------------------+----------------+
| feedback_id   | int(11)   | NO   | PRI | NULL               | auto_increment |
| rating        | int(11)   | YES  |     | 5                  |                |
| comments      | text      | NO   |     | NULL               |                |
| feedback_date | date      | NO   |     | NULL               |                |
| user_id       | int(11)   | NO   | MUL | NULL               |                |
| bus_id        | int(11)   | NO   | MUL | NULL               |                |
| route_id      | int(11)   | NO   | MUL | NULL               |                |
| submitted_at  | timestamp | NO   |     | current_timestamp() |                |
+---------------+-----------+------+-----+--------------------+----------------+
8 rows in set (0.06 sec)

mysql> desc routes;
+----------------+----------------------------------------+------+-----+-----------+----------------+
| Field          | Type                                   | Null | Key | Default   | Extra          |
+----------------+----------------------------------------+------+-----+-----------+----------------+
| route_id       | int(11)                                | NO   | PRI | NULL      | auto_increment |
| from_location  | varchar(100)                           | NO   |     | NULL      |                |
| to_location    | varchar(100)                           | NO   |     | NULL      |                |
| departure_date | date                                   | NO   |     | NULL      |                |
| departure_time | time                                   | NO   |     | NULL      |                |
| cost           | decimal(10,2)                          | NO   |     | NULL      |                |
| bus_id         | int(11)                                | NO   | MUL | NULL      |                |
| status         | enum('SCHEDULED','COMPLETED','CANCELLED') | YES  |     | SCHEDULED |                |
+----------------+----------------------------------------+------+-----+-----------+----------------+
8 rows in set (0.07 sec)
```

```
mysql> desc users;
+--------------+-----------------------------------+------+-----+--------------------+----------------+
| Field        | Type                              | Null | Key | Default            | Extra          |
+--------------+-----------------------------------+------+-----+--------------------+----------------+
| user_id      | int(11)                           | NO   | PRI | NULL               | auto_increment |
| first_name   | varchar(50)                       | NO   |     | NULL               |                |
| last_name    | varchar(50)                       | NO   |     | NULL               |                |
| email        | varchar(100)                      | NO   | UNI | NULL               |                |
| phone_number | varchar(15)                       | NO   | UNI | NULL               |                |
| password     | varchar(255)                      | NO   |     | NULL               |                |
| role         | enum('PASSENGER','ADMIN','AGENT') | YES  |     | PASSENGER          |                |
| created_at   | timestamp                         | NO   |     | current_timestamp() |                |
+--------------+-----------------------------------+------+-----+--------------------+----------------+
8 rows in set (0.06 sec)
```

DB CONNECTION SCRIPT

```
<?php

// ================================================================

// THE DATABASE PLUG-IN (CONNECTION CONFIGURATION)
```

```
// ==================================================================
// Think of this script like a "plug" that connects our website to the
// filing cabinet (Database) where all our data is stored.
// We include this file at the top of every page that needs to READ or WRITE data.
// ==================================================================

// --- STEP 1: ENABLE ERROR REPORTING ---
// When we are learning to code, mistakes happen!
// These two lines tell the computer: "If there is a mistake in my code,
// please show me a big message so I can find and fix it."
error_reporting(E_ALL);
ini_set('display_errors', '1');

// --- STEP 2: DEFINE THE FILING CABINET (DB) LOCATION ---
// $server_name: Where is the database? "localhost" means it's on the same computer as the code.
// $username: The ID used to enter the database. "root" is the default for XAMPP.
// $password: The secret key. By default in XAMPP, this is empty ("").
// $database_name: The specific folder/cabinet we created in MySQL.
$server_name = "localhost";
$username = "root";
$password = "";
$database_name = "IBBS_PROTOTYPE";
$port = 3306; // This is the default "door number" (port) that MySQL uses.

// --- STEP 3: ESTABLISH THE CONNECTION ---
```

```
// We create a new 'mysqli' object. This is like picking up the phone and calling the database.
$conn = new mysqli($server_name, $username, $password, $database_name, $port);

// --- STEP 4: CHECK IF THE CALL WAS SUCCESSFUL ---
// if ($conn->connect_error) checks if someone "picked up the phone".
if ($conn->connect_error) {
    // If it failed (like a busy signal), we stop everything (die) and show the error.
    die("Connection Failed: " . $conn->connect_error);
}

// If we reach this point, the connection is working!
// The $conn variable is now ready to be used to run SQL commands.
?>
```

CODE FOR SENDING DATA TO DATABASE

```
<?php
// This is the main "Account Engine" script. It handles two big jobs:
// 1. SIGNING UP: Letting new people create an account.
// 2. LOGGING IN: Letting existing people enter the site.

// We tell PHP to show all errors. This is very helpful for us as students because
// if something goes wrong, the computer will tell us exactly which line has a problem!
error_reporting(E_ALL);
ini_set('display_errors', '1');
```

```
// session_start() is like opening a user's personal notebook on the server.
// Anything we write in there (like their name or ID) will be remembered on every page they visit.
session_start();

// We need to talk to the database to save or find users, so we include our connection file.
require_once 'db_connection.php';

// ==================================================================
// PART 1: SIGNING UP (CREATE A NEW ACCOUNT)
// ==================================================================

// We check if the user clicked the button named 'save' in the signup form.
if (isset($_POST['save'])) {

    // We collect the information the user typed.
    // trim() removes any accidental spaces at the beginning or end of what they typed.
    $first_name = trim($_POST['first_name']);
    $last_name = trim($_POST['last_name']);
    $email = trim($_POST['email']);
    $phone_number = trim($_POST['phone_number']);
    $password_raw = trim($_POST['password']);

    // --- STEP A: SECURITY CHECK (PASSWORD STRENGTH) ---
    // We want to make sure passwords are hard to guess.
```

```
    // This 'preg_match' is a pattern checker. It makes sure the password has:
    // One big letter, one small letter, one number, and one symbol.
    if (!preg_match('/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*()\-_=+{};:,<.>]).{8,}$/', $password_raw)) {
        die("Password must contain at least one uppercase, one lowercase, one digit, one special character, and be at least 8 characters long.");
    }

    // --- STEP B: CHECK IF EMAIL IS REAL ---
    // filter_var helps us check if the email has an @ symbol and a proper structure.
    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        die("Invalid email address.");
    }

    // --- STEP C: LIMIT DOMAINS (Project Rule) ---
    // We only want to allow common email services like Gmail or Yahoo for this prototype.
    $allowed_domains = ["gmail.com", "yahoo.com", "hotmail.com"];
    $email_domain = explode("@", $email)[1]; // This grabs the part AFTER the '@'
    if (!in_array($email_domain, $allowed_domains)) {
        die("Email domain not allowed. Only Gmail, Yahoo, and Hotmail accepted.");
    }

    // --- STEP D: CHECK FOR DUPLICATES ---
    // We don't want two people using the same email or phone number.
    // We use a 'prepared statement' (?) to protect our database from hackers.
    $stmt = $conn->prepare("SELECT email FROM users WHERE email = ? OR phone_number = ?");
```

```
$stmt->bind_param("ss", $email, $phone_number);
$stmt->execute();
$stmt->store_result();

// If the computer found a row (num_rows > 0), it means the email/phone is already in the list!
if ($stmt->num_rows > 0) {
    $stmt->close();
    die("User with this email or phone number already exists!");
}
$stmt->close();

// --- STEP E: HASHING (SCRAMBLING) THE PASSWORD ---
// We NEVER save real passwords. Instead, we scramble them into a long code.
// Even if a hacker sees the database, they won't know the real password.
$hashed_password = password_hash($password_raw, PASSWORD_DEFAULT);

// Everyone who signs up through the public form starts as a "PASSENGER".
$role = "PASSENGER";

// --- STEP F: SAVE TO DATABASE ---
// Now we insert the new user into our 'users' table.
$stmt = $conn->prepare("INSERT INTO users (first_name, last_name, email, phone_number, password, role) VALUES (?, ?, ?, ?, ?, ?)");
// "ssssss" means we are providing 6 fragments of text (strings).
$stmt->bind_param("ssssss", $first_name, $last_name, $email, $phone_number, $hashed_password, $role);
```

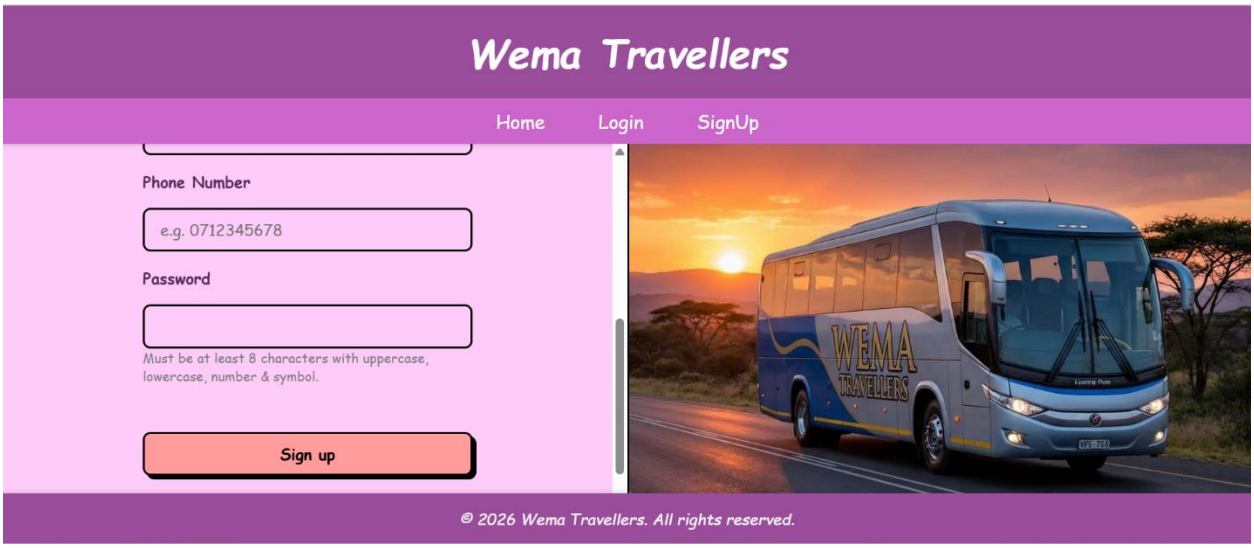
```
    if ($stmt->execute()) {
        // SUCCESS! Now we automatically log them in.
        $new_user_id = $stmt->insert_id; // Get the unique ID the database just made for them.

        // Write their info into our server notebook (Session).
        $_SESSION['user_id'] = $new_user_id;
        $_SESSION['role'] = $role;
        $_SESSION['name'] = $first_name . " " . $last_name;

        $stmt->close();

        // Send them to their new Dashboard!
        echo "<script>alert('Sign-up successful! Welcome to Wema Travellers.');
window.location.href='dashboard.php';</script>";
        exit();
    } else {
        $stmt->close();
        die("Signup failed: " . $conn->error);
    }
}

// ==================================================================
// PART 2: LOGGING IN (ENTER THE SITE)
// ==================================================================
```

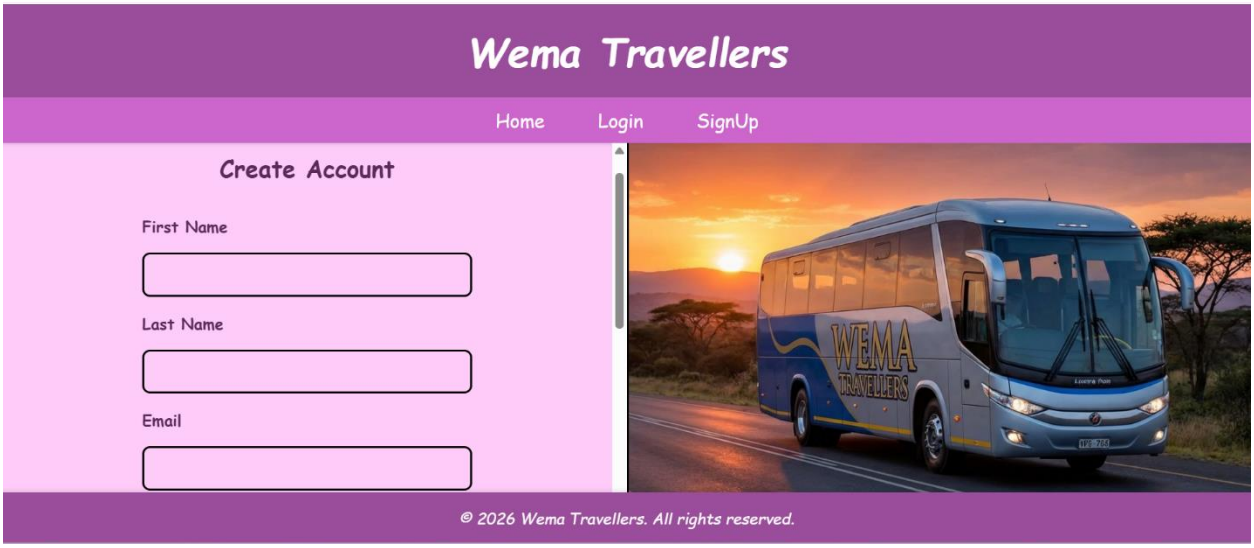
```
// We check if the user clicked the 'login' button.
if (isset($_POST['login'])) {

    // Collect what they typed in the login boxes.
    $email = trim($_POST['email']);
    $password_input = trim($_POST['password']);

    // --- STEP A: FIND THE USER ---
    // We ask the database: "Find the person who has this email."
    $stmt = $conn->prepare("SELECT user_id, password, role, first_name, last_name FROM users
WHERE email = ?");
    $stmt->bind_param("s", $email);
    $stmt->execute();
    $stmt->store_result();
    // Link the database results to our variables.
    $stmt->bind_result($user_id, $stored_password, $role, $f_name, $l_name);

    // If we found the email (rows > 0), now we check if they typed the right password.
    if ($stmt->num_rows > 0) {
        $stmt->fetch();

        // --- STEP B: CHECK PASSWORD ---
        // 'password_verify' compares the typed password with the scrambled one in the database.
        if (password_verify($password_input, $stored_password)) {
            // SUCCESS! Let's remember them in our notebook.
            $_SESSION['user_id'] = $user_id;
```

```
        $_SESSION['role'] = $role;
        $_SESSION['name'] = $f_name . " " . $l_name;
        $stmt->close();


        // --- STEP C: ROLE-BASED REDIRECTION ---
        // We send users to different places depending on their job (role).
        if($role == 'ADMIN') {
            // Administrators go to their secret Control Panel.
            echo "<script>alert('Login successful! Welcome Admin.');
window.location.href='admin_dashboard.php';</script>";
        } elseif ($role == 'AGENT') {
            // Agents go to their Workspace.
            echo "<script>alert('Login successful! Welcome Agent.');
window.location.href='agent_dashboard.php';</script>";
        } else {
            // Regular passengers go to their simple Dashboard.
            echo "<script>alert('Login successful!');
window.location.href='dashboard.php';</script>";
        }
        exit();
    } else {
        // WRONG PASSWORD: Tell them and send them back to try again.
        $stmt->close();
        echo "<script>alert('Incorrect password!'); window.location.href='login.html';</script>";
        exit();
    }
  } else {
```

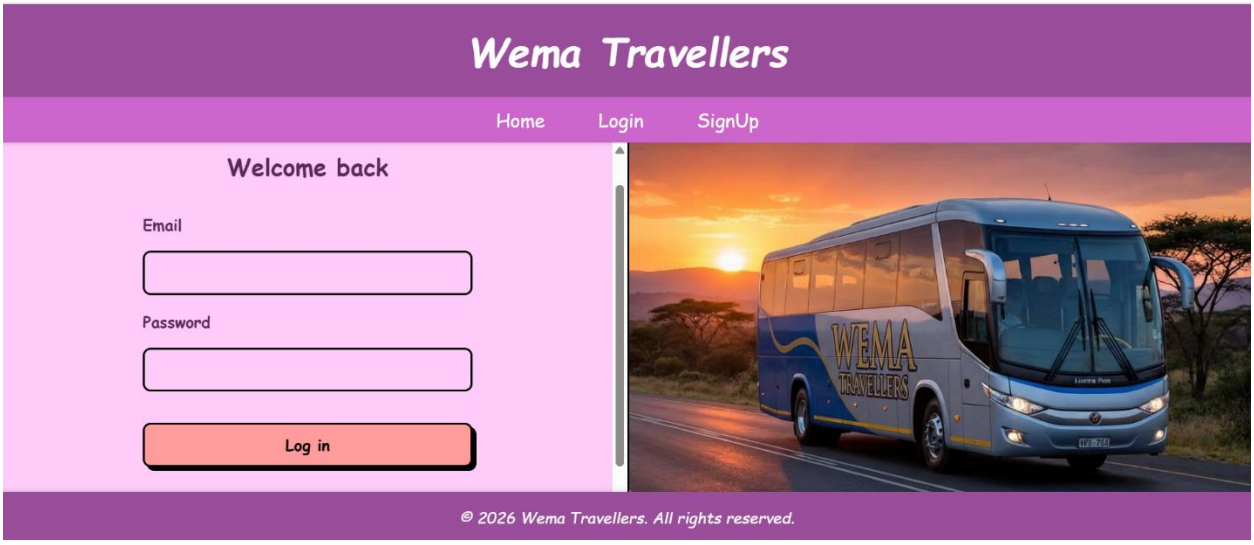
```
        // EMAIL NOT FOUND: If the email isn't in our list at all.
        $stmt->close();
        echo "<script>alert('Email not found!'); window.location.href='login.html';</script>";
        exit();
    }
}

// Close the database connection when the engine is finished.
$conn->close();
?>
```

SIGN UP PAGE

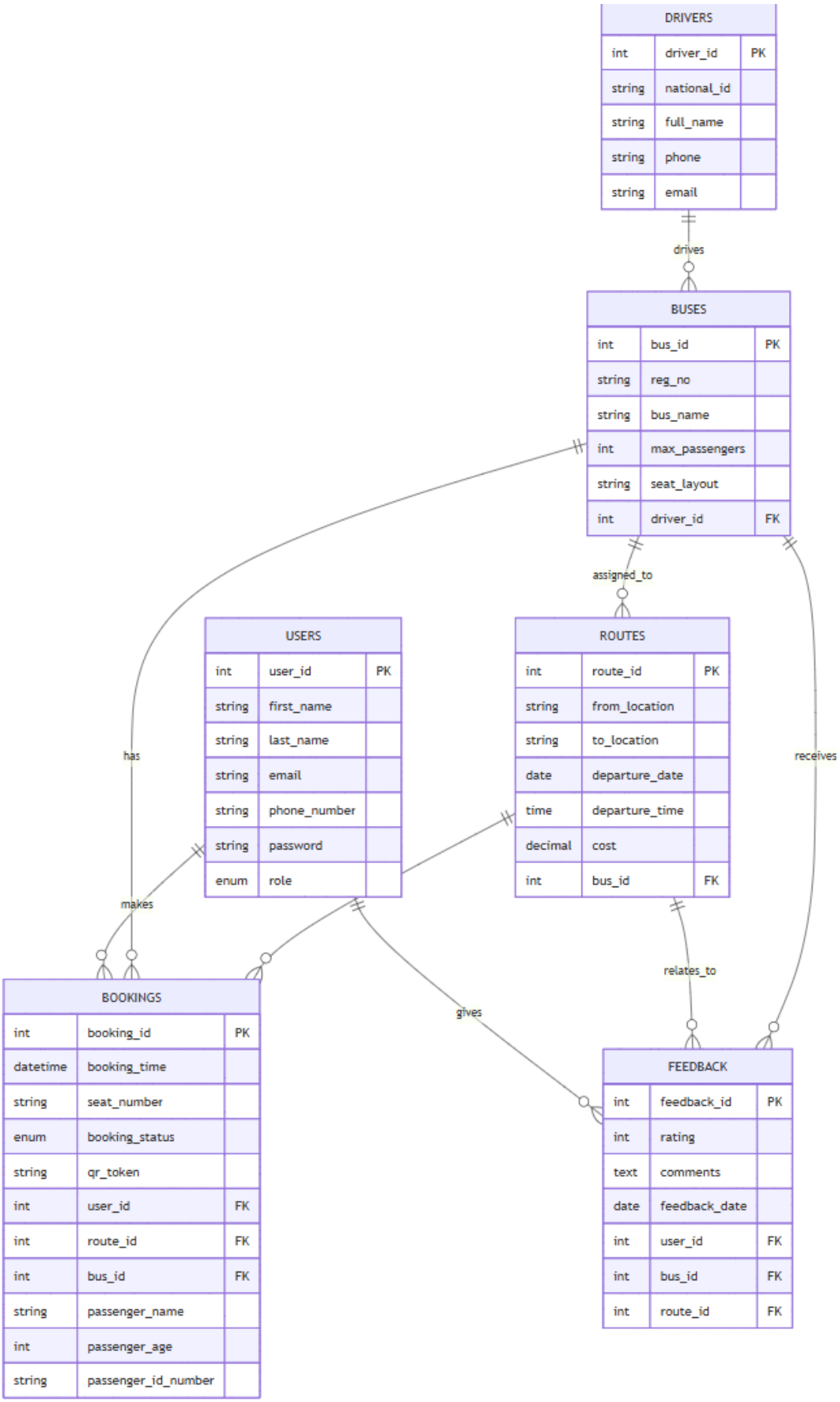




LOGIN FORM SNAPSHOT



ERD DIAGRAM


DRIVERS
int driver_id PK
string national_id
string full_name
string phone
string email
drives
BUSES
int bus_id PK
string reg_no
string bus_name
int max_passengers
string seat_layout
int driver_id FK
assigned_to
receives
has
makes
USERS
int user_id PK
string first_name
string last_name
string email
string phone_number
string password
enum role
ROUTES
int route_id PK
string from_location
string to_location
date departure_date
time departure_time
decimal cost
int bus_id FK
relates_to
gives
BOOKINGS
int booking_id PK
datetime booking_time
string seat_number
enum booking_status
string qr_token
int user_id FK
int route_id FK
int bus_id FK
string passenger_name
int passenger_age
string passenger_id_number
FEEDBACK
int feedback_id PK
int rating
text comments
date feedback_date
int user_id FK
int bus_id FK
int route_id FK

Relational Schema for IBBS Prototype

Below is the relational schema for the International Bus Booking System (IBBS) prototype database.

1. Users

Stores all user information including Passengers, Admins, and Agents.

- user_id (PK, INT, Auto Increment)
- first_name (VARCHAR, Not Null)
- last_name (VARCHAR, Not Null)
- email (VARCHAR, Unique, Not Null)
- phone_number (VARCHAR, Unique, Not Null)
- password (VARCHAR, Not Null)
- role (ENUM: 'PASSENGER', 'ADMIN', 'AGENT', Default: 'PASSENGER')

2. Drivers

Stores information about bus drivers.

- driver_id (PK, INT, Auto Increment)
- national_id (VARCHAR, Unique, Not Null)
- full_name (VARCHAR, Not Null)
- phone (VARCHAR, Unique, Not Null)
- email (VARCHAR, Unique, Not Null)

3. Buses

Stores information about the buses in the fleet.

- bus_id (PK, INT, Auto Increment)
- reg_no (VARCHAR, Unique, Not Null)
- bus_name (VARCHAR)
- max_passengers (INT, Not Null)
- seat_layout (VARCHAR, Not Null)

- driver_id (FK -> Drivers.driver_id, Nullable)

4. Routes

Stores available international travel routes.

- route_id (PK, INT, Auto Increment)
- from_location (VARCHAR, Not Null)
- to_location (VARCHAR, Not Null)
- departure_date (DATE, Not Null)
- departure_time (TIME, Not Null)
- cost (DECIMAL, Not Null)
- bus_id (FK -> Buses.bus_id, Not Null)

5. Bookings

Stores ticket bookings made by users.

- booking_id (PK, INT, Auto Increment)
- booking_time (DATETIME, Not Null)
- seat_number (VARCHAR, Not Null)
- booking_status (ENUM: 'PAID', 'CHECKED_IN', 'CANCELLED', Default: 'PAID')
- qr_token (VARCHAR, Unique, Not Null)
- user_id (FK -> Users.user_id, Not Null)
- route_id (FK -> Routes.route_id, Not Null)
- bus_id (FK -> Buses.bus_id, Not Null)

6. Feedback

Stores user feedback and ratings.

- feedback_id (PK, INT, Auto Increment)
- rating (INT, 1-5)
- comments (TEXT)
- feedback_date (DATE, Not Null)

- user_id (FK -> Users.user_id, Not Null)
- bus_id (FK -> Buses.bus_id, Not Null)
- route_id (FK -> Routes.route_id, Not Null)

Relationships

- **Users** have multiple **Bookings** (1:N).
- **Users** can leave multiple **Feedback** (1:N).
- **Drivers** are assigned to **Buses** (1:1 or 1:N, schema allows 1 driver per bus at a time).
- **Buses** have multiple **Routes** (1:N).
- **Routes** have multiple **Bookings** (1:N).
- **Routes** have multiple **Feedback** (1:N).